

QUESTÕES

Questão 1. Contagem de Trocas no Bubble Sort

O algoritmo **Bubble Sort** percorre repetidamente um vetor, comparando pares de elementos adjacentes e realizando uma troca (*swap*) sempre que eles estiverem fora de ordem.

Dado um vetor de N inteiros, execute o algoritmo Bubble Sort tradicional em ordem crescente e determine tanto o vetor ordenado final quanto a **quantidade total de trocas** realizadas durante todo o processo.

Entrada

A primeira linha contém um número inteiro N ($1 \leq N \leq 10^4$), representando o tamanho do vetor. A segunda linha contém N inteiros separados por espaço.

Saída

A primeira linha da saída deve conter os N inteiros ordenados em ordem crescente, separados por espaço. A segunda linha deve conter um único número inteiro representando o total de trocas realizadas.

Exemplo de Entrada	Exemplo de Saída
5 4 3 2 1 5	1 2 3 4 5 6
4 1 2 3 4	1 2 3 4 0

Explicação

- **Exemplo 1:** Para ordenar a sequência $[4, 3, 2, 1, 5]$, o algoritmo executa 6 trocas entre elementos adjacentes até que todos fiquem posicionados corretamente em $[1, 2, 3, 4, 5]$.
- **Exemplo 2:** Como a sequência de entrada já se encontra perfeitamente ordenada, nenhuma condição de troca é satisfeita, resultando em 0 trocas efetuadas.

Questão 2. Seleção dos k -Menores Elementos via Selection Sort

Uma propriedade fundamental do algoritmo **Selection Sort** é a sua invariante de laço: após k iterações do laço externo, os k menores elementos do vetor original já estão posicionados de forma definitiva e ordenada nos índices $0, 1, \dots, k - 1$.

Dado um vetor de N números inteiros e um valor inteiro k ($1 \leq k \leq N$), execute estritamente as primeiras k passagens do laço externo do Selection Sort e exiba os k primeiros elementos da lista resultante.

Entrada

A primeira linha contém dois inteiros N ($1 \leq N \leq 10^4$) e k ($1 \leq k \leq N$). A segunda linha contém N inteiros separados por espaço.

Saída

Imprima uma única linha contendo os k menores elementos selecionados e posicionados pelo algoritmo, em ordem crescente e separados por espaço.

Exemplo de Entrada	Exemplo de Saída
6 3 29 10 14 37 13 42	10 13 14
5 2 8 5 3 9 1	1 3

Explicação

- **Exemplo 1:**

- 1ª iteração ($i = 0$): O menor valor global (10) é colocado no índice 0.
- 2ª iteração ($i = 1$): Na sublista restante, o menor valor (13) vai para o índice 1.
- 3ª iteração ($i = 2$): O menor valor restante (14) vai para o índice 2.
- Os $k = 3$ primeiros elementos são 10 13 14.

- **Exemplo 2:** Após 2 iterações do Selection Sort, os dois menores valores do conjunto (1 e 3) ocupam as duas primeiras posições do vetor.

Aluno (a):

Questão 3. Ordenação de Vetor Quase-Ordenado (k -Sorted) com Insertion Sort

Um vetor é dito **k -ordenado** (*nearly sorted*) se cada um de seus elementos dista no máximo k posições da sua posição final correta no vetor ordenado. O algoritmo **Insertion Sort** é altamente eficiente para este cenário, operando em complexidade de tempo $\mathcal{O}(N \cdot k)$ em vez do pior caso quadrático $\mathcal{O}(N^2)$.

Dado um vetor de N inteiros que está k -ordenado, execute o Insertion Sort e contabilize o **número total de comparações** realizadas no laço interno (**while**) durante a inserção de cada elemento.

Entrada

A primeira linha contém dois inteiros N ($1 \leq N \leq 10^4$) e k ($0 \leq k \leq N$). A segunda linha contém N inteiros separados por espaço representando o vetor k -ordenado.

Saída

A primeira linha da saída deve conter o vetor totalmente ordenado em ordem crescente. A segunda linha deve conter um número inteiro indicando a quantidade total de comparações realizadas pelo laço do Insertion Sort.

Exemplo de Entrada	Exemplo de Saída
6 2 3 2 1 5 4 6	1 2 3 4 5 6 8
5 1 2 1 4 3 5	1 2 3 4 5 6

Explicação

- **Exemplo 1:** Cada elemento está a no máximo $k = 2$ posições de distância do seu local final. A cada inserção, poucas comparações e deslocamentos são necessários, totalizando apenas 8 comparações para ordenar todo o vetor.
- **Exemplo 2:** No vetor 1-ordenado, elementos adjacentes invertidos são ajustados com no máximo 1 ou 2 passos de recuo por elemento, totalizando 6 comparações.

Questão 4. Ordenação Paridade-Bifásica com Bubble Sort

Adapte a rotina de comparação do algoritmo **Bubble Sort** para organizar um vetor de inteiros sob um critério composto de ordenação com duas regiões:

1. Todos os números **pares** devem anteceder todos os números **ímpares**.
2. Os números pares devem ficar dispostos em ordem **crescente** entre si.
3. Os números ímpares devem ficar dispostos em ordem **decrescente** entre si.

Entrada

A primeira linha contém um inteiro N ($1 \leq N \leq 10^4$). A segunda linha contém N inteiros separados por espaço.

Saída

Imprima uma única linha contendo os N elementos ordenados conforme o critério de paridade e ordem solicitado, separados por espaço.

Exemplo de Entrada	Exemplo de Saída
8 5 2 9 8 1 6 3 4	2 4 6 8 9 5 3 1
5 7 11 3 1 9	11 9 7 3 1

Explicação

- **Exemplo 1:**

- Números pares: 2, 8, 6, 4 $\rightarrow$ ordenados crescentemente: [2, 4, 6, 8].
- Números ímpares: 5, 9, 1, 3 $\rightarrow$ ordenados decrescentemente: [9, 5, 3, 1].
- Resultado final: 2 4 6 8 9 5 3 1.

- **Exemplo 2:** Como o vetor não possui números pares, todos os elementos ímpares são dispostos em ordem decrescente: 11 9 7 3 1.

Questão 5. Contagem de Inversões via Simulação do Insertion Sort

O número de **inversões** de um vetor A é definido como a quantidade de pares de índices (i, j) tais que $i < j$ e $A[i] > A[j]$. Essa métrica mede diretamente o grau de desordem de uma sequência e coincide com o número exato de deslocamentos à direita executados pelo algoritmo **Insertion Sort** para transformar o vetor de entrada em uma lista ordenada.

Dado um vetor de N inteiros positivos distintos, simule a execução do Insertion Sort e determine a quantidade mínima de deslocamentos/inversões necessários para ordená-lo completamente em ordem crescente.

Entrada

A primeira linha contém um inteiro N ($1 \leq N \leq 10^4$), representando a quantidade de elementos do vetor. A segunda linha contém N inteiros distintos separados por espaço.

Saída

Imprima um único número inteiro representando a quantidade total de inversões (deslocamentos realizados no Insertion Sort).

Exemplo de Entrada	Exemplo de Saída
5 2 4 1 3 5	3
5 5 4 3 2 1	10

Explicação

- **Exemplo 1:** No vetor $[2, 4, 1, 3, 5]$, as inversões existentes são: $(2, 1)$, $(4, 1)$ e $(4, 3)$. Logo, o número total de inversões (e deslocamentos no Insertion Sort) é igual a 3.
- **Exemplo 2:** Para um vetor em ordem estritamente decrescente de tamanho $N = 5$, o total de inversões corresponde ao pior caso do Insertion Sort: $\frac{N(N-1)}{2} = \frac{5 \times 4}{2} = 10$.

”Algorithms + Data Structures = Programs.”
– Niklaus Wirth